

Lecture 2: Setting Up C# & Writing Your First Program

- [Lecture 2: Setting Up C# & Writing Your First Program](#)
 - [1. What Do You Need to Write C# Code?](#)
 - [2. Choosing Your Tools](#)
 - [3. Installing the .NET SDK](#)
 - [Step-by-Step \(All Platforms\)](#)
 - [Why “SDK” and not just “Runtime”?](#)
 - [4. Installing Visual Studio Code + C# Extension](#)
 - [5. Creating Your First C# Project](#)
 - [Step 1: Create a new console project](#)
 - [Step 2: Move into the project folder](#)
 - [Step 3: Look at what was generated](#)
 - [6. Understanding the Generated Files](#)
 - [Program.cs \(Modern C# — .NET 6+\)](#)
 - [The “Classic” version \(what’s really happening underneath\)](#)
 - [HelloWorldApp.csproj](#)
 - [7. Running Your Program](#)
 - [What Actually Happens Behind the Scenes](#)
 - [8. Modifying Your First Program](#)
 - [9. Common Beginner Errors \(and How to Read Them\)](#)
 - [10. Quick Recap \(Key Takeaways\)](#)
 - [11. What’s Next?](#)

Lecture 2: Setting Up C# & Writing Your First Program

Picking up where Lecture 1 left off — this lecture is 100% hands-on. By the end, you’ll have a working C# development environment and you’ll have written, run, and understood your very first C# program.

1. What Do You Need to Write C# Code?

To write and run C# code, you need three things:

1. **The .NET SDK** – This is the actual engine. It contains the C# compiler, the runtime, and the command-line tools (`dotnet`) that build and run your programs.
2. **A Code Editor / IDE** – A place to write your code comfortably (with syntax highlighting,

autocomplete, error checking).

3. **A Terminal / Command Line** – To run commands like `dotnet build` and `dotnet run`.

Important distinction: The **.NET SDK** is what makes your code actually run. The editor (Visual Studio, VS Code, Rider) is just a convenient tool for *writing* the code — it doesn't compile or run anything by itself.

2. Choosing Your Tools

Tool	Type	Best For	Platform
Visual Studio (Community Edition)	Full IDE	Beginners, Windows users, large enterprise/game projects	Windows, macOS (limited)
Visual Studio Code	Lightweight code editor + extensions	Cross-platform development, lightweight projects	Windows, macOS, Linux
JetBrains Rider	Full IDE (paid, free for students)	Professional developers wanting speed & smart tooling	Windows, macOS, Linux
.NET SDK (CLI only)	Command-line only	Learning fundamentals, servers, automation	Windows, macOS, Linux

Recommendation for beginners: Start with **Visual Studio Code + .NET SDK**. It's free, lightweight, cross-platform, and teaches you what's actually happening (unlike Visual Studio, which can hide a lot of the process behind wizards and templates).

3. Installing the .NET SDK

Step-by-Step (All Platforms)

1. Go to the official Microsoft .NET website and download the **latest LTS (Long-Term Support) SDK** — currently **.NET 8** is the LTS version most beginners should install.
2. Run the installer for your operating system (Windows `.exe`, macOS `.pkg`, or Linux package manager).
3. Verify the installation by opening a terminal and typing:

```
dotnet --version
```

If installed correctly, this prints something like `8.0.100`.

Why “SDK” and not just “Runtime”?

- The **Runtime** only lets you *run* already-built C# applications.
 - The **SDK** includes the Runtime **plus** the compiler and project tools needed to *build* applications.
 - As a learner, you always want the **SDK**, not just the runtime.
-

4. Installing Visual Studio Code + C# Extension

1. Download and install **Visual Studio Code** from the official site (free, cross-platform).
2. Open VS Code, go to the **Extensions** panel (the four-squares icon on the left sidebar).
3. Search for and install:
 - **C# Dev Kit** (official Microsoft extension — includes IntelliSense, debugging, project management)
 - **.NET Install Tool** (usually installed automatically as a dependency)
4. Restart VS Code after installation.

Tip: You do *not* need Visual Studio (the full IDE) to learn C#. VS Code + C# Dev Kit is enough for the entire beginner-to-intermediate journey, and it's what many professional .NET developers use daily on Linux/macOS.

5. Creating Your First C# Project

C# code doesn't just live as a loose `.cs` file — it lives inside a **project**, which is managed by a `.csproj` file. The `dotnet` CLI creates this structure for you automatically.

Step 1: Create a new console project

```
dotnet new console -o HelloWorldApp
```

- `dotnet new console` → tells the SDK to create a new **console application** template.
- `-o HelloWorldApp` → the `-o` (output) flag names the folder `HelloWorldApp`.

Step 2: Move into the project folder

```
cd HelloWorldApp
```

Step 3: Look at what was generated

```
HelloWorldApp/  
├─ HelloWorldApp.csproj    ← project configuration file  
├─ Program.cs             ← your actual C# code lives here  
└─ obj/                   ← auto-generated build files (ignore for now)
```

6. Understanding the Generated Files

Program.cs (Modern C# — .NET 6+)

```
Console.WriteLine("Hello, World!");
```

That's it. Modern C# (since .NET 6) uses what's called **top-level statements** — you don't need to manually write a `class` or a `Main` method for a simple program; the compiler generates that scaffolding behind the scenes.

The “Classic” version (what's really happening underneath)

Before .NET 6, and still useful to understand, every C# program was structured like this:

```
using System;

namespace HelloWorldApp
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello, World!");
        }
    }
}
```

Part	Meaning
using System;	Imports the <code>System</code> namespace, which contains basic classes like <code>Console</code> .
namespace HelloWorldApp	Groups related code together under one name (avoids naming conflicts).
class Program	C# is object-oriented — everything must live inside a class.
static void Main(string[] args)	The entry point of every C# application — execution always starts here.
Console.WriteLine(...)	Prints text to the console/terminal, followed by a new line.

Why this matters: Even though modern C# hides this structure for simple programs, the compiler is still generating a class and a `Main` method internally. Understanding the classic form means you'll never be confused when you open older code, tutorials, or enterprise codebases — almost all real-world production C# still uses the explicit `class / Main` structure.

HelloWorldApp.csproj

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net8.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>
</Project>
```

Setting	Meaning
OutputType=Exe	This project builds into an executable program (not a library).
TargetFramework=net8.0	Which version of .NET this project targets.
ImplicitUsings=enable	Automatically imports common namespaces so you

	don't write <code>using System;</code> everywhere.
<code>Nullable=enable</code>	Turns on modern null-safety warnings (helps prevent a huge class of bugs).

7. Running Your Program

From inside the project folder, run:

```
dotnet run
```

Output:

```
Hello, World!
```

What Actually Happens Behind the Scenes

1. `dotnet run` triggers the **C# compiler (Roslyn)** to translate your `.cs` files into **IL (Intermediate Language)** — a CPU-independent bytecode.
2. The IL is packaged into a `.dll` /executable inside the `bin/` folder.
3. The **CLR (Common Language Runtime)** loads that IL and **Just-In-Time (JIT) compiles** it into native machine code for your specific CPU.
4. The machine code executes, and you see the output.

This is exactly the “Compiled Language” behavior described in Lecture 1’s comparison table — C# is compiled to IL first, then JIT-compiled at runtime, giving it a balance of portability (the IL runs on any OS with .NET installed) and speed (native machine code execution).

8. Modifying Your First Program

Let’s make it more interactive. Open `Program.cs` and replace the contents with:

```
Console.WriteLine("Hello, World!");
Console.Write("What is your name? ");
string name = Console.ReadLine();
Console.WriteLine($"Nice to meet you, {name}! Welcome to C#.");
```

Run it again with `dotnet run`, type your name, and press Enter.

Line	What It Does
<code>Console.Write(...)</code>	Prints text without a new line (cursor stays on the same line).
<code>Console.ReadLine()</code>	Pauses the program and waits for the user to type input, then press Enter.
<code>string name = ...</code>	Declares a variable named <code>name</code> and stores the user’s input in it.
<code>\$"Nice to meet you, {name}!"</code>	A string interpolation — the <code>\$</code> before the string

lets you embed variables directly using `{ }`.

This one small program already demonstrates four core building blocks you'll study in depth in the next lectures: **output**, **input**, **variables**, and **string formatting**.

9. Common Beginner Errors (and How to Read Them)

Error Message	What It Usually Means	Fix
CS0103: The name 'X' does not exist in the current context	You used a variable/method before declaring it, or misspelled it	Check spelling and declaration order
CS1002: ; expected	You forgot a semicolon at the end of a statement	Every C# statement ends in <code>;</code>
CS0246: The type or namespace 'X' could not be found	Missing <code>using</code> statement or typo in a class name	Add the correct <code>using</code> or fix the name
command not found: dotnet	The SDK isn't installed or not in your system PATH	Reinstall the SDK and restart your terminal

Tip for beginners: C# error messages almost always tell you the **exact line number** and a **CS error code**. Get comfortable reading them — searching the exact error code (e.g., “CS1002”) is one of the fastest ways to fix issues as a beginner.

10. Quick Recap (Key Takeaways)

- You need the **.NET SDK** (compiler + runtime + tools) and a code editor (VS Code recommended) to start writing C#.
- `dotnet new console -o ProjectName` scaffolds a new project; `dotnet run` builds and executes it.
- Modern C# uses **top-level statements** to hide boilerplate, but every program is still built around a **class** and a **Main entry point** underneath.
- `Console.WriteLine`, `Console.Write`, and `Console.ReadLine` are your first tools for **output** and **input**.
- C# code is **compiled to IL**, then **JIT-compiled to native machine code** by the CLR at runtime — this is what makes it both portable and fast.

11. What's Next?

In Lecture 3, we'll dive into: 1. Variables and data types (`int`, `double`, `string`, `bool`, `char`) 2. Type conversion and casting 3. Constants and naming conventions 4. Basic arithmetic and operators

Tip for students: Before moving on, try creating three different console projects and printing

three different personalized messages using `Console.ReadLine()` and string interpolation. Muscle memory matters more than reading at this stage.